ALGORITHM WHITEPAPER

MiniDB Studio

순수 C 코어, 규칙 기반 query planning, B+ Tree traversal, WAL recovery, 그리고 상태 기반 desktop observability를 정리한 이중 언어 스토리지 엔진 백서.

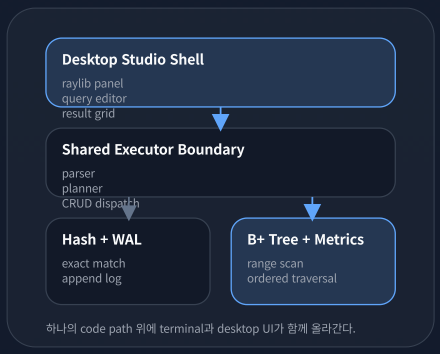
이 문서는 현재 코드베이스에 구현된 MiniDB Studio의 실제 구조를 정리한다. 재사용 가능한 engine module, index-aware query execution, deterministic assistant suggestion, 그리고 동일한 executor boundary 위에서 동작하는 raylib desktop synchronization을 그대로 설명한다.

- Pure C11 Core
- Hash Index
- B+ Tree
- CSV Snapshot
- WAL Replay
- raylib Desktop

엔진 경계	인덱스 경로	복구 모델	문서 세트
재사용 가능	3	CSV + WAL	영문 + 국문

아키텍처 스냅샷

공유 execution core 덕분에 terminal과 desktop surface 모두에서 query semantic, recovery rule, telemetry가 동일하게 유지된다.



대상 독자	문서 범위	포지셔닝

엔지니어링 포트폴리오 검토자, 면접관, 시스템 프로그래밍 동료.

parser, planner, execution engine, recovery, desktop synchronization, performance telemetry.

설명 가능한 내부 구조와 데모 친화적 UI를 갖춘 lightweight database management studio.

섹션 02

Project Overview

MiniDB Studio는 in-memory record engine, rule-based optimizer, 그리고 desktop control surface를 하나의 일관된 구조로 묶는다. 단순 parser 데모가 아니라 작은 database studio처럼 동작하도록 설계된 포트폴리오 프로젝트다.

시스템 범위

- `C11` 기반 storage core가 record, index, recovery state, execution statistic의 수명을 직접 관리한다.
- 터미널 콘솔과 raylib desktop app은 동일한 `parse -> plan -> execute` 경계를 재사용한다.
- 정확 일치 조회는 hash index로 가속하고, ordered access와 range scan은 B+ Tree가 담당한다.
- Persistence는 snapshot + log replay 모델로 구성해, 복구 흐름을 짧은 면접 설명 안에서도 명확하게 전달할 수 있다.

모듈 구성

모듈	주요 책임	대표 S
`parser.c`	tokenization, validation, command decoding.	`parse`
`planner.c`	규칙 기반 access-path 선택과 sort pushdown 판단.	`plann`
`database.c`	CRUD dispatch, filter evaluation, index maintenance, query statistic 집계.	`datab` `datab`
`bptree.c`	ordered indexing, split/rebalance logic, range traversal.	`bptre` `bptre`
`wal.c`	append-only mutation log와 restart replay.	`wal_a`

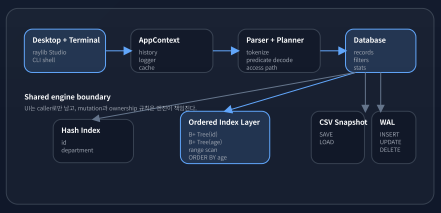
모듈	주요 책임	대표 S
`studio_app.c`	desktop state transition, result grid behavior, metric streaming.	`studi` `studi`

섹션 03

Core Storage Engine Architecture

가장 중요한 설계 결정은 ownership 분리다. Record의 생명주기는 `Database` 안에 고정되고, 보조 index는 `Record*` 참조만 빌려 쓴다. UI에 전달되는 summary는 별도의 임시 result row를 materialize하여 desktop layer가 storage memory의 owner가 되지 않도록 유지한다.

실행 계층



기존 terminal과 desktop studio가 동일한 execution path를 공유한다. 이 분리 덕분에 pure C engine은 유지하고 presentation layer만 확장할 수 있다.

Memory Ownership Model

이 ownership 분리 덕분에 LOAD도 안전하다. 임시 `Database`를 먼저 완성하고 검증한 뒤에만 `AppContext`에 교체할 수 있다.

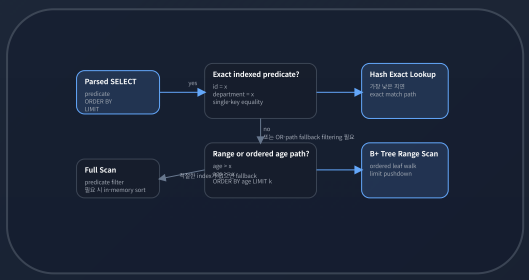
> 설계 invariant: mutation path가 index와 persistence metadata를 먼저 갱신하고, 그 다음에 UI가 새로운 상태를 읽는다. raylib layer는 storage object를 직접 free하지 않는다.

섹션 04

Query Planner & Optimizer Algorithm

Optimizer는 의도적으로 lightweight하고 explainable하다. cost model 대신 query shape에 따라 최적 access path를 고르는 deterministic rule set을 사용하며, 선택된 plan 문자열은 그대로 desktop inspector에 표시된다.

Planner Decision Flow



Planner는 `B+ Tree Range Scan`, `Hash Exact Lookup` 같은 plan label을 생성하고, 이 문자열이 그대로 desktop inspector에 노출된다.

규칙 집합과 Planner 효과

QUERY SHAPE	PRIMARY PLAN	PLANNER EFFECT
`SELECT WHERE id = 42`	Hash exact lookup 또는 B+ exact lookup	하나의 bucket 또는 key slot만 검사하므로 row scan 수가 매우 작다.
`SELECT WHERE department = Oncology`	Hash exact lookup	department hash index를 seed path로 사용하고 후보 chain만 추가 평가한다.
`SELECT WHERE age > 30`	B+ Tree range scan	첫 qualifying leaf에서 시작해 linked leaf chain을 따라간다.
`SELECT ORDER BY age LIMIT 10`	B+ Tree ordered scan	leaf traversal 결과가 이미 정렬되어

QUERY SHAPE	PRIMARY PLAN	PLANNER EFFECT
		있으면 추가 sort를 생략한다.
혼합 AND predicate	Indexed seed + post-filter	가장 좋은 seed path를 선택한 뒤 나머지 조건을 in-memory로 평가한다.
광범위한 OR 또는 미지원 조합	Full scan	정확성을 우선해 전체 record walk로 fallback한다.

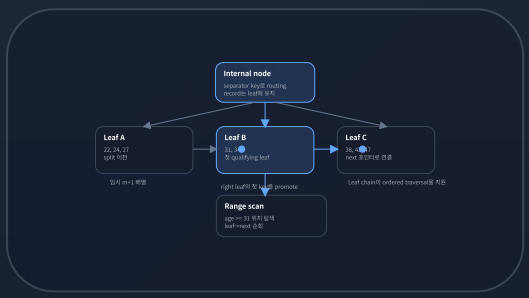
Planner는 추가 in-memory sort가 필요한지도 함께 결정한다. leaf traversal이 이미 `ORDER BY age` 를 만족하면 reorder 비용을 피할 수 있다.

섹션 05

B+ Tree Traversal Logic

Ordered index layer의 목적은 age 기반 range scan과 ordered traversal을 단순 선형 검색이 아니라 storage-engine 수준의 동작으로 보이게 만드는 것이다. 실제 record는 모두 leaf에 유지하고, internal node는 separator key만 보관하며, leaf는 forward chain으로 연결된다.

Split and Range Path



현재 구현은 `bptree.c` 안에 split comment를 직접 남겨 leaf/internal promotion 규칙이 코드 리뷰에서도 바로 보이도록 했다.

Operational Semantics

- `bptree_insert`는 올바른 leaf까지 내려간 뒤 정렬 순서를 유지하며 삽입하고, overflow가 발생했을 때만 split 결과를 상위로 전파한다.
- Leaf split은 여유 슬롯이 하나 있는 임시 배열을 사용해 lower half는 현재 node에 남기고 upper half는 새로운 right sibling으로 이동한 뒤, 그 right sibling의 첫 key를 promote한다.
- Internal split은 가운데 separator를 상위로 promote하면서 child 배열을 재배치해 parent routing을 유지한다.
- 중복 age는 하나의 leaf key 아래 value list로 연결되므로, duplicate key를 위해 separator를 반복 생성하지 않아도 된다.
- Delete는 borrow-or-merge rebalancing helper를 사용해 underflow node를 복구한 뒤 caller로 돌아간다.

OPERATION	TYPICAL COMPLEXITY	의미
Exact lookup	O(log_B n)	separator 따라 하나 leaf로 내리

OPERATION	TYPICAL COMPLEXITY	의미
Range scan	O(log_B n + k)	한 번 seek leaf chain 스트리밍한
Insert	O(log_B n)	overflow node에서 split 발생.
Delete	O(log_B n)	underflow borrow/m

섹션 06

WAL and Snapshot Recovery Flow

Persistence는 checkpoint 가능한 snapshot과 append-only mutation log 조합으로 모델링된다. 이 방식은 디스크 동작을 이해하기 쉽다. SAVE는 안정적인 CSV 이미지를 만들고, LOAD는 그 이후 mutation을 WAL에서 replay한다.

Recovery Sequence

Executor는 먼저 임시 `Database`에 CSV를 로드하고 `data/db.log`를 replay한 뒤에만 active context를 교체한다.

Recovery Guarantees

단계	현재 알고리즘	의미
Mutation logging	`INSERT|...`, `UPDATE|...`, `DELETE|...` 레코드를 append.	마지막 snapshot 이후 모든 변경이 재생 가능한 형태로 남는다.
Snapshot save	모든 row를 CSV로 기록한 뒤 WAL을 checkpoint.	CSV가 간결한 restore point가 되고 log 길이도 제어된다.
Restore	임시 engine에 CSV를 load하고 WAL을 replay하며 record insert 과정에서 index를 함께 복원.	정상 실행과 동일한 code path로 index 일관성이 복원된다.
Activation	성공 후에만 restored database를	부분 복구 상태가 active

단계	현재 알고리즘	의미
	`AppContext`에 대입.	session을 오염시키지 않는다.

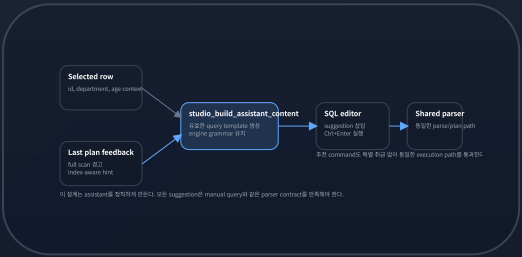
Replay가 normal database mutation API를 그대로 호출하기 때문에, index consistency는 별도의 recovery 단계가 아니라 insert/update/delete 재사용의 결과로 자연스럽게 유지된다.

섹션 07

AI Query Assistant Parsing Flow

현재 desktop assistant는 의도적으로 deterministic하다. LLM runtime이 아니라, row context와 이전 execution plan, 그리고 demo heuristic을 바탕으로 유효한 SQL-like suggestion을 생성한 뒤, 그 suggestion을 사용자가 입력한 text와 동일한 parser 경로로 다시 흘려보낸다.

Current Assistant Pipeline



예를 들어 row 선택이나 plan feedback을 바탕으로 `SELECT WHERE id = ...`, `SELECT WHERE age > ... ORDER BY age LIMIT` 같은 query를 제안할 수 있다.

Assistant Design Contract

- 선택된 row 정보가 deterministic query template의 seed가 되므로, 현재 grammar에서 항상 유효한 문장만 생성한다.
- 직전 execution plan이 full scan이었다면 더 index-friendly한 대안을 우선 제안한다.
- Editor가 항상 visible source of truth로 남기 때문에, 사용자는 실행 전에 suggestion을 직접 확인하고 수정할 수 있다.
- Assistant output도 `parse_command`를 다시 통과하므로 validation, logging, benchmarking, caching이 그대로 유지된다.

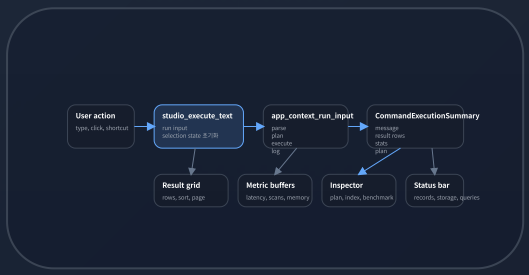
실제 natural-language parsing은 roadmap에 두는 것이 맞다. 현재 구현은 engine state를 context-aware query suggestion으로 바꿔 주는 형태의 “AI assistant”를 제공한다.

섹션 08

Desktop UI-State Synchronization

Desktop application이 반응성을 유지하는 이유는 UI state가 얇고 파생적이기 때문이다. Query execution은 `AppContext` 내부에서 engine state를 갱신하고, desktop layer는 editor, grid, inspector, metric dashboard, footer에 필요한 summary만 소비한다.

State Transition Graph



Desktop shell은 storage view를 직접 재구성하지 않는다. 대신 compact execution summary와 dashboard status snapshot을 받아 필요한 패널만 갱신한다.

Synchronization Points

UI EVENT	주요 함수
Query 실행	`studio_execute_text`
Engine dispatch	`app_context_run_input`
Metrics update	`studio_push_metrics`

UI EVENT	주요 함수
Grid sort	`result_grid_sort`
Footer refresh	`app_context_snapshot_status`

이 흐름 덕분에 studio는 시각적으로 풍부해도 별도의 storage layer가 되지 않는다. UI는 presentation state를, engine은 truth를 가진다.

섹션 09

Performance Dashboard Metrics

MiniDB Studio는 engine 내부 동작을 presentation-friendly telemetry로 드러낸다. 각 command 실행 후 compact metric series를 갱신하므로, desktop dashboard는 결과뿐 아니라 결과가 어떤 비용 구조로 만들어졌는지도 설명할 수 있다.

Metric Data Flow

Dashboard는 이미 execution summary에 존재하는 engine-side signal을 시각화한다. 즉 UI가 값을 만들어내는 것이 아니라, 엔진이 집계한 telemetry를 읽는다.

Metric Definitions

METRIC	SOURCE FIELD
Query latency trend	`last_elapsed_ms`
Rows scanned	`summary.stats.rows_scanne`
Memory usage	`database_estimate_memory_`
Cache hit ratio	`query_cache_hits / query_cache_lookups`

METRIC	SOURCE FIELD
Index usage frequency	chosen plan 기준 counter

Dashboard는 의도적으로 lightweight하다. 작은 chart buffer만 유지하고, 별도 telemetry subsystem 없이 execution accounting 결과를 그대로 보여 준다.

섹션 10

Complexity Analysis

이 엔진은 asymptotic 관점에서 설명하기에 충분히 작고, exact lookup 구조와 ordered traversal, recovery 작업 사이의 trade-off를 보여 주기에 충분히 깊다. 아래 차트는 리뷰어가 기대해야 할 상대적 비용 프로파일을 요약한다.

Relative Cost Profile

정렬이 중요한 query에서는 point lookup보다 B+ Tree range scan이 더 좋은 path가 될 수 있다. 비용 구조가 다르기 때문이다.

Complexity Table

OPERATION	COMPLEXITY	의미
Tokenize + parse	O(L)	query tex 선형이므로 end 동작이 가능하다.
Hash exact lookup	O(1) average	id 또는 departme equality f 가장 강력하
B+ Tree exact lookup	O(log_B n)	작은 tree 안정적인 c search를
B+ Tree range scan	O(log_B n + k)	한 번 seek leaf chair streaming
Insert / delete with index maintenance	O(log_B n)	record lookup/a 이후에는 B maintena 지배 비용이
Snapshot load	O(n)	모든 row가 normal in path로 다 구성된다.

OPERATION	COMPLEXITY	의미
WAL replay	O(m log_B n)	replay 길 index reb 비용에 비
Result-grid sort fallback	O(r log r)	planner가 order를 노 못할 때만

섹션 11

Future Scaling Roadmap

현재 구조는 의도적으로 compact하지만, 이미 더 큰 storage-engine 작업으로 확장될 seam을 갖고 있다. 아래 roadmap은 pure C core를 유지하면서 disk management, richer planning, 그리고 더 강한 assistant behavior로 확장하는 방향을 정리한다.

NEAR-TERM

Local engine depth

slotted-page serialization, richer index statistic, 더 정교한 predicate normalization을 추가하되 현재 in-memory API surface는 유지한다.

MID-TERM

Storage realism

buffer-pool abstraction, asynchronous checkpointing, composite index, 더 명시적인 recovery metadata를 도입해 SAVE/LOAD를 page-oriented persistence로 확장한다.

LONG-TERM

DB studio evolution

기존 parser boundary를 유지한 채 MVCC, cost-based optimizer, validated natural-language-to-query conversion을 올린다.

Roadmap Principle

다음 단계로 갈수록 현재 프로젝트의 장점은 더 강화되어야 한다. explainable internals, reusable boundary, 그리고 실제 engine behavior를 시각화하는 UI가 핵심이다.

섹션 12

Final Engineering Summary

MiniDB Studio가 포트폴리오 시스템으로 설득력을 가지는 이유는 engine, optimizer, recovery path, desktop shell이 하나의 일관된 서사를 만들기 때문이다. 몇 분 안에 설명할 수 있을 만큼 compact하고, indexing, ownership, execution trade-off를 깊게 이야기할 만큼 충분히 rich하다.

왜 이 설계가 일관적인가

REUSABLE CORE

동일한 parser, planner, database code가 console과 desktop studio를 함께 구동한다. 그래서 구조가 "복제된 UI"가 아니라 의도된 architecture로 보인다.

EXPLAINABLE PERFORMANCE

Hash lookup, B+ Tree traversal, scan count, cache hit가 모두 눈에 보이므로 하나의 query path를 실제 엔진 수준에서 설명할 수 있다.

RECOVERY CLARITY

CSV snapshot + WAL replay 조합은 durability, restore

포트폴리오 포지셔닝

MiniDB Studio는 시스템 프로그래밍 면접에서 좋은 인상을 주는 판단을 보여준다. explicit ownership rule, data-structure-aware query planning, predictable recovery, 그리고 low-level decision을 읽기 쉽게 드러내는 presentation layer가 핵심이다.

20초 데모 포인트: pure C storage core, hash + B+ Tree indexing, WAL-backed recovery, shared executor boundary, 그리고 query plan과 runtime metric을 시각화하는 desktop studio.

safety, index reconstruction을 명확한 mental model로 정리해 준다.